Ares-Flash: Efficient Parallel Integer Arithmetic Operations Using NAND Flash Memory

Jian Chen
Tsinghua University
Beijing, China
jian-che21@mails.tsinghua.edu.cn

Congming Gao
Xiamen University
Xiamen, China
gaocm@xmu.edu.cn

Youyou Lu
Tsinghua University
Beijing, China
luyouyou@tsinghua.edu.cn

Yuhao Zhang
Tsinghua University
Beijing, China
zhangyuhao@mail.tsinghua.edu.cn

Jiwu Shu
Tsinghua University
Beijing, China
shujw@tsinghua.edu.cn

2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO) | 979-8-3503-5057-9/24/$31.00 ©2024 IEEE | DOI: 10.1109/MICRO61859.2024.00109

***Abstract*—In-Flash Processing (IFP) has been proposed in recent years to realize computation ability inside NAND flash memory. Distinguished from processing-in-memory (PIM) and in-storage-processing (ISP), IFP reduces the data movement starting from the most bottom flash memory medium. It is especially beneficial to those applications that require high processing parallelism (e.g., huge databases, large-scale image processing, etc.). However, IFP works often require expensive extra hardware modification, which brings lots of energy dissipation and area overhead. Recent works, Parabit and Flash-Cosmos, try to implement calculations using flash memory with minimum hardware modification. But their accomplishments still remain at the level of simple bitwise operations, and thus it prevents their work from being widely adopted.**

In this work, we propose Ares-Flash, a new technology using flash memory to support more complex integer arithmetic operations (e.g., addition, accumulation, and multiplication). Ares leverages the designed page buffer to perform basic mechanisms: full-adder logic and bit-shift. Moreover, we construct computational operations using dedicated control sequences in the page buffer upon two basic mechanisms. Our experimental results indicate that Ares is highly efficient and significantly mitigates data movement from storage to memory or computing units (e.g., CPUs, GPUs). Quantitatively, Ares averagely improves performance and energy efficiency by 18.58×/9.89× and 97.9×/14× compared to the out-storage-processing(OSP)/in-storage-processing(ISP) under accumulation tasks with real workloads. It also improves 8.2×/4.5× and 89×/13.2× when performing vector-vector multiplication on real-world workloads.

***Index Terms*—In-memory computing, 3D NNAD, style, flash memory, near data processing.**

I. Introduction

In the era of big data, we are witnessing the rise of various applications that generate and utilize vast amounts of data [8], [16], [78]. Within these data-intensive applications, addition and multiplication are ubiquitous and fundamental operations. Therefore, it is necessary for modern computing systems to support high-performance and energy-efficient parallel arithmetic operations (i.e., add, mul).

One primary bottleneck to perform efficient and energy-friendly computations in traditional computing architecture is the data movements between memory and computing units. For example, in GPUs, the process of bulk multiplication incurs significantly less latency compared to reading large amounts of data from storage. To address this issue, Near Data Processing (NDP) has been proposed to tackle the data transfer problem via offloading computation near memory/storage devices [12], [46], [55]–[58], [79], [83]. Therefore, only the computation results are transferred over the bus, greatly reducing data movement and energy costs.

Among various NDP architectures, we identify that IFP (In-Flash-Processing) is one of the most efficient approaches. Data-intensive applications often store large amounts of data in flash-based SSDs, the most common storage medium due to its large capacity and high speed [**?**], [70]. IFP moves computation closer to the flash medium compared to other NDP techniques like Processing-in-Memory (PIM) and In-Storage-Processing (ISP). The latter still face data movement overhead spent on I/O bus (i.e., PCIe [2]) and device internal bus (i.e., ONFI [6], [36]) respectively, making limited performance gain. However, to facilitate complex computation, traditional IFP approaches utilize analog-to-digital-converter(ADC) and extra computing elements. This either leads to increased area and energy dissipation [12], [24], [37], [53], [67] or decreased parallelism since intrinsic high BL-level parallelism is sacrificed [43].

To solve such problem, previous efforts, ParaBit [33] and Flash-Cosmos [72], aimed to enable the computation ability **using NAND flash memory**. ParaBit achieves this by adjusting the sequence of sensing operation in MLC flash and controlling signals of the latch circuit such that the operands are ingeniously migrated between latches to get the final result from the output latch. Flash-Cosmos leverages the electrical characteristics in the NAND block to realize `AND` and `OR` operations by applying sensing voltage on multiple wordlines simultaneously. Both works successfully offload computations into flash memory with minimum hardware modification while maintaining high-level parallelism. Nevertheless, they remain limited in their application scope as their functionality is

979-8-3503-5057-9/24/$31.00 ©2024 IEEE
DOI 10.1109/MICRO61859.2024.00109

restricted to basic bitwise operations.

We aim to support more applicable arithmetic operations using flash memory. We identify that the page buffer in the advanced 3D-NAND flash memory is an appropriate computational platform for us because it provides us with ample operational space and sufficient BL-level parallelism. However, unlike bitwise operations in previous works, where only two bits from one BL are involved, we have to consider each bit from all BLs at once. This makes it difficult to deploy the calculating process because computing processes have dependencies, while all BLs must be operated concurrently. Additionally, while existing bitwise operations can construct adder logic, the foundation of arithmetic operations, both [33] and [72] only perform bitwise operations on data stored within the cell. This approach necessitates multiple reads and programs to obtain AND/OR/XOR results, with intermediate results serving as inputs for subsequent operations. Moreover, transferring carry-in values using internal bus is unavoidable. Together, these factors result in significantly higher latency compared to directly reading raw data. Thus, to implement applicable complex operations in flash, we have to mainly tackle two challenges: (1) **Efficient Adder Logic**: A suitable approach is needed compared to existing works to form the adder logic more efficient. While all BLs will simultaneously perform the same operations, each bit's addition in the adder has a sequential dependency. It is crucial to ensure that the n-bit adder operates correctly across all BLs, achieving both time and space efficiency. (2) **Transfer-free Bit-Shift**: Implementing bit-shifts in both addition and multiplication without using the internal bus for data transfer is a significant hurdle. This requires a strategy to perform shifts efficiently within the flash memory's constraints and support multiple data types at the same time.

In this work, we propose Ares-Flash (Ares), a new technology that supports integer arithmetical operations using flash memory. Different from previous approaches [33], [72] that perform bitwise operations during the sensing process, Ares performs all computations inside the page buffer. We first make minor modifications to the page buffer's latch circuits, allowing for more flexible and efficient bitwise operations so that we can optionally reuse input and output operands. Then, we orchestrate control signals sequence in the page buffer, reusing intermediate values and overlapping part of the workflow. This approach minimizes the number of control steps in the full adder logic across all BLs while using the fewest possible latches. Furthermore, to perform efficient bit-shift, we design an extra transmission peripheral in the page buffer, supporting data types ranging from 4-bit to 32-bit. The transmission peripheral offloads indispensable data transfer (each bit shift) inside flash memory without going through internal channel. Upon these, Ares hierarchically supports addition and accumulation, where accumulation is achieved by consecutively executing multiple addition operations to obtain the final sum. For multiplication, which comprises accumulation and bitwise AND operations, Ares introduces specific flash commands with corresponding control scheme

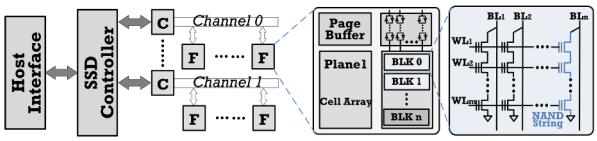


Fig. 1: NAND Flash SSD. C:Flash Controller. F:Flash Chip.

to enable efficient scalar multiplication operation and reduce internal data transfer for input data. Other operations, like bulk matrix-matrix products, can be composed by proposed operations. In short, the contributions of this work are:

1) We introduce Ares-Flash, which efficiently performs integer arithmetic operations using NAND flash with 0.84% extra area overhead. To our knowledge, this is the first IFP work that supports general advanced computations (e.g., add, multiply) without any extra computing hardware.
2) We propose Ares-based SSD design, including new APIs and additional components for implementing Ares-Flash.
3) We evaluate Ares using real-world workloads, and the results demonstrate that Ares brings significant performance and energy benefits compared to other architectures.

II. Background

A. 3D NAND Flash Memory

3D NAND Flash Overview. NAND flash SSD is now the most prevalent storage medium across various applications due to its high speed and capacity [11], [30], [61], [70]. Figure 1 shows the overview architecture of modern NAND flash memory-based SSD. An SSD controller communicates with the host through a host interface (e.g., PCIe), and it is also in charge of flash chip management. The controller is composed of a flash translation layer (FTL), error checking and correction (ECC), garbage collection (GC), and other components. Flash chips are connected to the controller through the internal channel (internal bus, e.g., ONFI) with 8/16-bit width per transmission cycle. There can be 4-16 internal channels, and each attaches multiple flash chips. Each flash chip comprises one or two dies, each containing two or four planes. That is, NAND flash memory-based SSD supports four levels of parallelism (channel, chip, die, plane), and the controller can send requests to different flash planes and do operations simultaneously [20], [32], [34], [48], [73], [74], [84].

Inside NAND Flash Chip. Figure 1 depicts the internal architecture of a flash chip with 1 die that contains 2 planes; each plane is configured with page buffers and many memory blocks. Cells are vertically and horizontally stacked inside the block. The vertically stacked cells are serially organized as a string and connected to the bitline (BL). Flash cells in the horizontal direction are grouped as a wordline (WL). All bits with the same bit position in a WL make up one page, which is the smallest unit when reading/programming data.

Read Operation. When reading data from flash cells, the predefined sensing voltage V_{Ref} is applied to the target WL. At the same time, other WLs are applied with V_{Pass} that is higher than the maximum voltage value (i.e., 6V) so that the sensed

current passes through other cells. Such a process makes cells in the same string behave as an open circuit or a close circuit based on the voltage value of the sensed flash cell. Finally, the page buffer will get the sensing results (0 or 1) depending on the corresponding string's behavior.

Page Buffer. As shown in Figure 1, the Page buffer is primarily composed of multiple latches for each BL. Figure 2 gives a possible circuit design [70], where SO is the sensing node between the capacitor and the BL. M_n are transistors controlled by the chip controller. M_1- M_4 are used during program/read operations in L1, M_5 is used when transferring data from L1 to L2. Latch circuits play a significant role in both flash write and flash read operations. During a flash write operation, data needs to be loaded into the latch circuit first, and during a flash read operation, the read results are retrieved and stored in the latch circuit. As each cell may represent multiple bits, modern TLC/QLC (3/4-bit per cell) flash can have 5-6 latches in each latch circuit per BL, which are mainly used during program and verification. [42], [51], [80]

B. PIM, ISP and IFP

In traditional Von-Neumann architecture with data-intensive applications, data movement among storage, memory, and computing units has been one of the main performance bottlenecks. To mitigate this transfer overhead, processing-in-memory (PIM), in-storage-processing (ISP), and In-Flash-processing (IFP) are introduced.

PIM offloads some operations inside memory so that, instead of all operands, only computation results are transferred to computing units [7], [25], [31], [55]–[57], [63], [64], [68], [79], [89], [90]. It mitigates data transfer between memory and CPUs/GPUs, but it fails to reduce data movement from storage.

ISP processes data inside storage devices to avoid colossal data movement overhead from storage device to memory. It processes data on the controller side after reading data from the storage medium. Studies and products often use embedded ARM cores, FPGAs, and other specialized hardware to offload computations tasks without transferring raw data to the host [46], [58], [65], [81], [83]. However, ISP still reads massive raw data from storage mediums through the internal bus.

IFP performs computations inside flash memory before transferring data. It leverages the intrinsic high parallelism of SSD. Many IFP works leverage expensive extra hardware (e.g., ADCs or computational components) to do complex computations dedicated to specific applications (e.g., DNNs) [24], [37], [43], [47], [53], [66]. When reading data, the

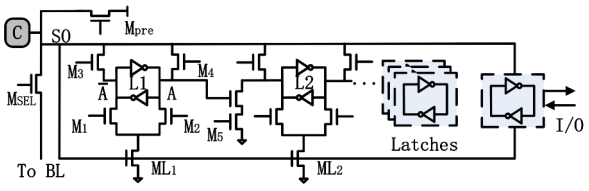


Fig. 2: Page buffer Per BL Example.

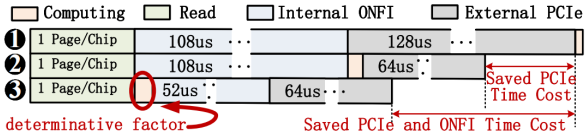


Fig. 3: Addition Latency: ❶compute outside SSD, ❷ISP with FPGA/ARM, ❸IFP

voltage/current values through BLs are translated into results by using the components. On the other hand, prior IFP studies, ParaBit [33] and Flash-Cosmos [72] propose to utilize flash memory with little hardware modification to perform bitwise operations. ParaBit leverages MLC latch circuits and uses multiple reads to get the final results. For example, it does two consecutive reads without initializing steps to perform logical-AND. Flash-Cosmos senses multiple WLs simultaneously to achieve bulk-bitwise operations.

III. Motivation

A. Addition using OSP, ISP and IFP

Addition and multiplication are fundamental operations in many data-intensive applications. They can further form up the accumulation, vector-vector multiplication, matrix multiplication, which are used in many fields like database, neural networks and machine learning. Since most of these applications can aggregate data thus reducing data movement, we infer applying IFP is an energy-efficient approach with high-performance. In Figure 3, we compare the latency of OSP(outside storage processing), ISP, and IFP when performing addition with configuration in Table I. Note that, OSP refers to all computing technologies outside the storage device, including PIM, CPU based computing. If we use OSP architecture, the computation happens after all data transfers, which is ❶. When we use ISP, the computations happen after the internal transfer, and the data volume through the external PCIe is halved, which is ❷. For OSP, both internal and external transfers bottleneck the overall performance; for ISP, the internal transfer is the main bottleneck. IFP can address this problem by performing computations before any data transfer. In ❸, both internal transfer and external transfer are reduced, which costs the lowest latency than other architectures. But the final benefit achieved by IFP is still determined by its computing efficiency.

B. Limitations of Existing IFP Works

In the following, we delve deeper into the implementation of the addition operation using different types of IFP solutions, including IFPs using hardware modifications and IFPs using intrinsic flash memory.

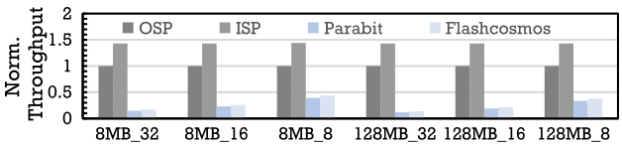


Fig. 4: Performance of Addition

IFPs using hardware modifications: IFP works frequently necessitate supplementary circuit components, including ADCs to leverage high BL-level parallelism and perform complex operations. Despite performance enhancements in certain instances, there are two main shortcomings. First, they are not readily adaptable to diverse applications. Once flash chips with extra computing units are manufactured, their computational functions are fixed. For example, flash chips equipped with ADC designed for 4-bit MAC are only suitable for accelerating certain quantized neural networks, but not for other applications. Second, IFP with hardware modifications may hinder the fundamental storage functionality of flash memory. For instance, the extra hardware in [66] requires 137% additional chip area and consumes more than 253× the chip power. This means that the chip area for standard storage is squeezed, and the device becomes less energy efficient.

IFPs using intrinsic flash memory: ParaBit and Flash-Cosmos leverage BL-level parallelism without expensive hardware modification, but they are constrained to rudimentary bitwise operations, which limits their applicability in broader contexts. Even though bitwise operations are integral to any other computational processes at the circuit level, we show that ParaBit and Flash-Cosmos are insufficient for constructing more sophisticated operations like addition through their bitwise processing. We use 8M and 128M pairs of data operands to perform addition, and all data operands are stored in the flash SSD. Figure 4 shows the overall calculation speed of addition by using OSP, ISP, ParaBit, and Flash-Cosmos with configuration in table I. In ParaBit and Flash-Cosmos, multiple internal transfers through the internal bus are used to transfer carry-in; multiple sensing and programming are used to perform bitwise operations and binary full-adder. The results reveal that ParaBit and Flash-Cosmos are inefficient when performing addition. For the int32 data type, the addition throughput of ParaBit and Flash-Cosmos are a mere 11% and 13%, respectively, of the OSP that utilizes the host CPU for addition. For the int8 data type, their performance is confined to just 33% and 37% of the OSP. This significant performance gap is mainly attributed to their increased reliance on the internal bus for transferring carry-in, which takes more than 40% of the total latency. Furthermore, they require more flash read and even write operations to perform addition because their operation process relies on sensing during reads and cannot optionally reuse input and output.

Our goal is to build a practical and versatile IFP architecture that supports arithmetic operations using intrinsic flash memory. We believe ParaBit and Flash-Cosmos have not fully developed the potential of flash structures. In this work, (i) we propose page buffer designs with controls to perform suitable and flexible binary operations. (ii) Then, we propose new in-flash-processing techniques that provide energy-efficient and high-performance addition-based operations (including accumulation and multiplication, then other functions like matrix-multiplication and vector-multiplication can be achieved).

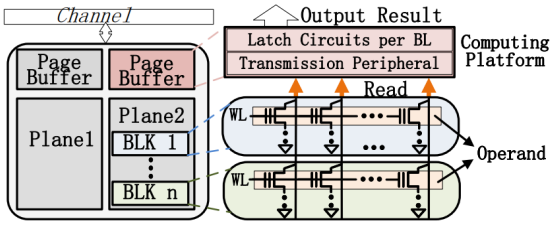


Fig. 5: Ares-based Flash Chip Overview.

IV. Design of Ares-Flash

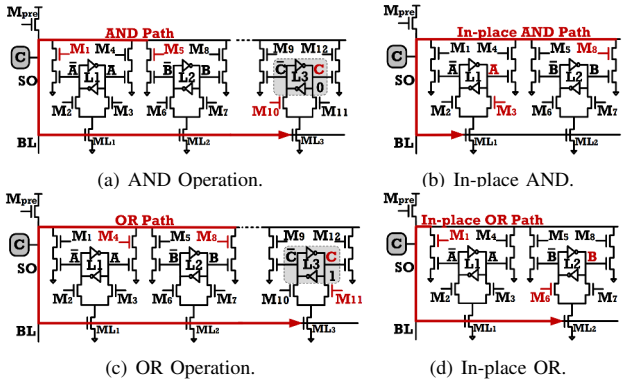
A. Ares-Flash Overview

Ares-Flash supports basic addition, accumulation, and multiplication, and other operations are achievable through combinations of these functions. We use page buffer as our computing platform (Figure 5), and the page buffer for each BL is composed by five latches with little hardware modification. Inside the page buffer, we design efficient adder logic with minimum latency and use transmission peripherals to mitigate data transfer. Before using Ares operations, users have to store their data inside NAND flash memory with Ares programming interface. When getting computation results, flash controllers will first send Ares operational commands to flash chips. Then, after data is read from NAND flash cells, all operations fundamentally happen inside the page buffer. Finally, only results go through the internal channel to the SSD controller.

We first describe how we conduct basic adder logic with designed page buffer. Then, we propose calculating approaches, including accumulation and multiplication, based on adder logic. Finally, we provide Ares-based SSD implementation.

B. Page Buffer and Basic Operations

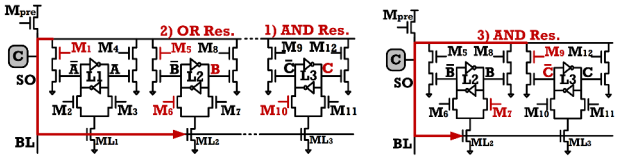
We design the latch circuits per BL in the page buffer to support basic bitwise operations as shown in Figure 6. The upper transistors (e.g., M_1, M_4 for L1) are tasked with setting SO to the value in L1. Conversely, the lower transistors (e.g., M_2, M_3 for L1) are responsible for setting the L1 based on SO. Importantly, this design is non-intrusive concerning the flash memory's conventional read and programming functions. For example, in read operations, M_{pre} is activated to initialize SO. Subsequently, M_3 is engaged to prepare L1 for storing



(a) AND Operation. (b) In-place AND.

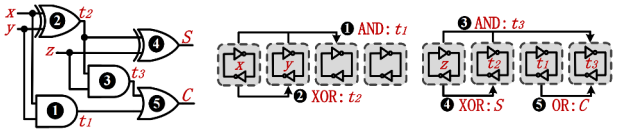
(c) OR Operation. (d) In-place OR.

Fig. 6: Latch Circuits design in Page Buffer with Operations.



(a) First Two Steps of XOR. (b) Third Step of XOR.

Fig. 7: XOR Implementation in Latch Circuit.



(a) 1-bit Full Adder. (b) Workflow in Latch Circuit.

Fig. 8: 1-bit Full Adder Implementation in Latch Circuit.

binary 1. In the course of the sensing phase, M_2 is actuated; if BL is an open circuit, the value in L1 retains the binary 1; if BL is a close circuit, L1 switches to binary 0, which represent the sensing results. Diverge from methodologies that integrate computations within the sensing phase, we introduce four novel and flexible types of basic bitwise operations within the latch circuits, thus expanding the potential applications. M_{pre} is enabled to set SO high at every beginning.

AND operation is engineered to output the logical conjunction of inputs A and B (stored in L1 and L2). There are two variants of the AND operation in Ares. In Figure 6(a), the first type of AND operation is performed, and the result is assigned to an available latch that is initialized with value 0. This is accomplished by concurrently activating M_1 and M_5, which maintains SO at high voltage if both A and B stores 1. If not, either M_1 or M_5 will ground the SO. Following this, M_{10} is enabled to store the result in a free latch L3.

In-place AND operation illustrated in Figure 6(b) retains the result in the original latch L1. This process involves sequential activation of M_8 and M_3. If B is 1, SO remains high, setting L1 to 1. If A is 1, activating M3 does not alter its state to 1. The logical AND result 1 is produced only when both A and B are 1. The outcome can also be stored in L2 by sequentially activate M_4 and M_7.

OR operation works to select larger value of input A and B and can be implemented into two types as well. The first type of OR operation portrayed in Figure 6(c) deposits the result into a free latch L3 that is initialized with 1 by enabling M_{10}. First, we enable M_4 and M_8 simultaneously that ensures SO to be grounded if one of the A and B stores 1. Then the M_{11} is enabled to use SO to reset L3's value, resulting in L3 keeping to store 1 when either A or B is 1.

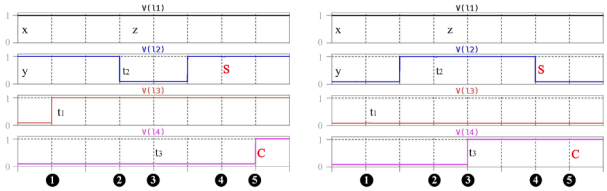
In-place OR operation is presented in Figure 6(d), we serially enable M_1 and M_6 to store result in L2. When M_1 is activated, SO equals A. Upon M_6's activation, if A (SO) is 1, the value in L2 is set to 1 as high SO signal will ground $\overline{B}$; if A (SO) is 1, the value of B is preserved. As a result, we get the result of the in-place OR operation in L2. The result can also be stored in L1 by sequentially activate M_5 and M_3.

With these four types of operations, we can not only conserve the integrity of input and output values within the page buffer but also optimize space by reusing latches. For instance, we store the result in a free latch when both inputs are needed for further operations. And we can store the result in the original latch to save space if an input becomes redundant. Moreover, we extend our approach to perform additional bitwise operations within the same rationale. For bitwise NAND and NOR, we can read the reverse bit value by activating opposite transistors (e.g., M_4 is opposite to M_1). For bitwise **XOR**, we use the combination of bitwise AND and bitwise OR as elucidated in Figure 7. As (A $\oplus$ B) can be expressed as $(\overline{A \wedge B}) \wedge (A \vee B)$. We first perform A AND B and store the result in a free latch L3. Next, we conduct an in-place OR operation and store the result in L2. Finally, we conduct an in-place AND while one input is the reverse value. Instead of how we perform standard in-place AND in Figure 6(b), in Figure 7(b), we activate M9 to reverse its input. All three steps of XOR operation are finished, and the result is stored in L2.

C. 1-bit Full Adder Inside Page Buffer

We use four latches in page buffer per BL to orchestrate 1-bit full adder. The logic of 1-bit full adder can be decomposed into five operations steps as depicted in Figure 8(a). We perform these five operations step by step in the latches we design, as shown in Figure 8(b). Now we have latches L1-L4 inside the page buffer per BL, and the inputs x, y are stored in L1 and L2. In step ❶, AND operation is performed, and the result t_1 is stored in L3. In step ❷, XOR operation is executed, and the result t_2 stored in the L2. Since XOR also require AND during the process, we can reuse t_1 from the previous step, which reduces XOR process time. Next, we load the third input, z, into L1. Just like the first steps, in step ❸, t2 AND z is performed, and the result is stored in the L4. Then ❹, t2 XOR z is performed, and the result S is stored in L2. Finally, in step, ❺, t1 OR t3 is performed, and the result C is stored in L4. In binary addition, S is the sum result, and C is the carry-out to the higher digit.

To validate the correctness, we use Spice simulator to model the workflow, and the two examples are provided in Figure 9. In Figure 9(a), two inputs, x, y, and the carry-in z, are all 1. The result S and C are both 1 and stored in L2 and L4,



(a) x y z = 1 1 1. (b) x y z = 1 0 1.

Fig. 9: Spice waveform for 1-bit full adder verification in latch circuits. *x*, *y*, *z* are three inputs for 1-bit full adder. x-axis is the timeline for each step as Figure 8(b).

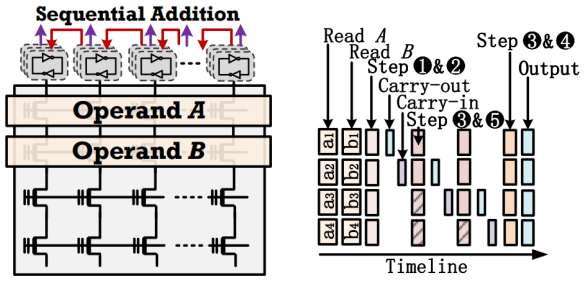
respectively. In Figure 9(b), x, y are 1 and 0, and the carry-in z is 1. The results S and C are 0 and 1, stored in L2 and L4.

D. Addition with Intra-Plane Transmission

Although we have implemented 1-bit full adder logic in the latch circuit per BL, two major problems exist in accomplishing binary addition inside flash memory. First, all BLs with latch circuits are operated concurrently in flash memory, but the addition process has steps of propagation from low digits to high digits. Ensuring correct calculations across all digits simultaneously with minimal steps is a complex task. Second, the addition is an N-bit full adder at the circuit level. This means each digit's calculation depends on the carry-in bit from the calculation of the previous digit. While simply using the internal bus to transfer carry-in increases huge operating latency, presenting a substantial hurdle. To solve these problems, we minimize the addition steps for Ares and design an extra transmission peripheral for bit-shift as follows:

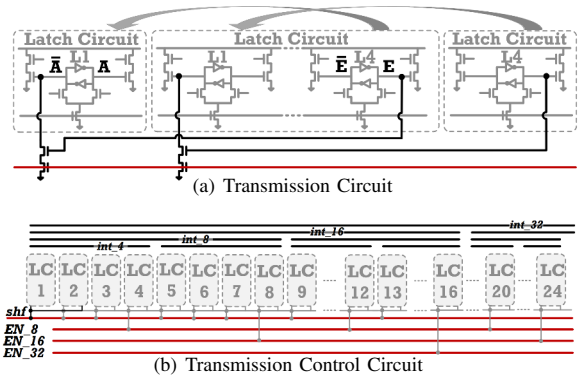
Addition Steps in Ares: In Ares-based addition, the data allocation is depicted in Figure 10(a). Each operand data is stored in the same WL, the same as standard data layout. Two operands are stored in different WLs but aligned on the same BL. Now we have to read two operand data and calculate from the least significant digit (LSD) to the most significant digit (MSD). After computing the LSD, the carry-out for the next digit and the sum result is prepared. However, since all BLs are operated simultaneously, this can disrupt the data on other BLs' latches. Meanwhile, reading data repeatedly for each digit's calculation will bring much more latency.

To solve the above problems, we rearrange the calculation steps for N-bit full adder in Ares addition, where the carry-in values of all digits are calculated, and then the sum result calculations of all digits are executed simultaneously. As the intermediate results t_1 and t_2 are independent from the carry-in calculation, we can safely keep t_1 and t_2 until the corresponding calculation is activated. The process of addition steps is presented in Figure 10(b), where we initiate the process by reading two input operands into L1 and L2 within each BL. Then we perform steps ❶ and ❷ to derive t_1 and t_2 which are stored in L3 and L2 across all digits. Next, we perform steps ❸ and ❺, which compute carry-out C and stores it into L4. Then we transfer carry-out to the next digit as the



(a) Data Layout. (b) Ares Timeline.

Fig. 10: Data Layout and Timeline for Ares-based Addition.



(a) Transmission Circuit

(b) Transmission Control Circuit

Fig. 11: Intra-plane Transmission Circuits

input and repeat these steps. During these steps, t_1 and t_2 are remained in L3 and L2 and do not change. This iterative procedure continues until we obtain the accurate carry-in for the MSD. At this point, steps ❸ and ❹ are executed in all BLs simultaneously to get the sum results, which then is stored in L2. With this approach, we have all the correct sum results and can transfer them to the host.

Transmission peripheral for Ares: To realize effective bit transmission and avoid transfer via the internal bus, we design an intra-plane transmission peripheral as illustrated in Figure 11. This setup in Figure 11(a) allows the carry-out from a lower digit be sent to a latch in the BL calculating a higher digit without using internal channels. As the carry-out bit has been calculated and stored in latch L4, we connect L4 to the inverse end of L1 of the next BL with two additional transistors. Before executing transmission, L1 is re-initialized to store 0 for variable A. Subsequently, the red line depicted in this figure is activated to facilitate bit transmission. If the value in the L4 (value E) equals 1, $\overline{A}$ is grounded, hence forcing the value of A to be 1. Conversely, if the value of E is 0, the value of A remains unaltered (A=0). Thus, we can transmit the value from the latch with lower digits to the latch calculating higher digits.

Given that the data size is smaller than the number of BLs, multiple data operands can be stored in the same WL and operated concurrently. Therefore, it is crucial to cut off the connection between adjacent data to maintain their isolation. Otherwise, the carry-out from one MSD could erroneously propagate to another calculation group, leading to incorrect results. For this, we introduce a transmission control circuit depicted in Figure 11(b). Our design so far supports operand sizes of 4, 8, 16, and 32 bits. Initially, transmission circuits are implemented within every 4 BL, effectively supporting 4-bit operation. To accommodate 8-bit, 16-bit, and 32-bit operations, the transmission between specific pairs of latch circuits is controlled using independent enable signals. For a 4-bit addition, only the shf signal is enabled. For an 8-bit addition, enabling not only shf, but also EN_8 signal to support the transmission. When extending the operand to 16 bits, the shf, EN_8 and EN_16 signals are all activated. Finally, for a 32-bit operand, all enable signals are activated.

The same rationale can be used for other data types.

Circuit Validation: To ensure the successful operation of the extra peripheral circuit design in the page buffer, we utilized SPICE simulation with Samsung industrial-compatible CMOS under 65nm process to validate its core operations. All transistors (inverters are not included) are 400nm gate length and 400nm gate width, and V_{cc} is 2V. Figure 12 demonstrates an example of the addition process of two 4-bit operands. Other data types are similar to 4-bit and can be derived from this example, so detailed examples for them are not provided here. Two operands A (1001) and B (0111) are initially stored in L1 and L2. The final sum results are stored in L2 with the final carry-out in 1-L4 (10000) at the end. During the operation, L4 stores the carry-out for the next digit, which is successfully transferred to L1 for the next digit. The simulation confirms that the designed page buffer with transmission peripherals functions correctly.

E. Accumulation

Accumulation significantly compresses data volume, which is suitable for IFP implementation. Ares performs accumulation by sequentially reading data and activating addition. After the first addition operation, the sum result is stored in the L2 of each BL. Then, the next data operand is read into L1 and performs another addition. When the computation involves *M* operand data (each bearing *N* bits), a total of *M-1* addition cycles are required for Ares-based addition.

F. Multiplication

Multiplication can be functioned using shift and accumulation operations, which have been supported in Ares. To illustrate this, consider there are two operands, denoted as A (represented by binary form $\{0010\}$ as an example) and B (represented by binary form $\{0110\}$) involved. The multiplication process is illustrated in Figure 13(a), which depicts operand A undergoing a shift operation four times. For each shift, an AND operation is performed between the shifted version of A and the corresponding bit of operand B. Finally, the results of these AND operations are cumulatively added to obtain the final product.

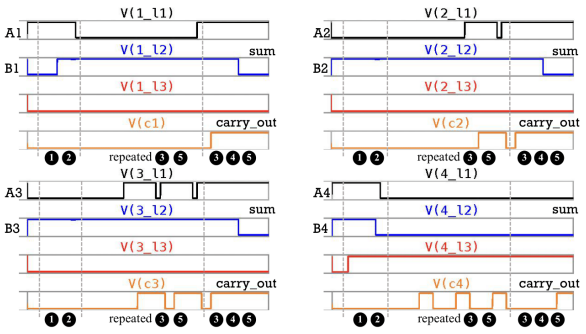
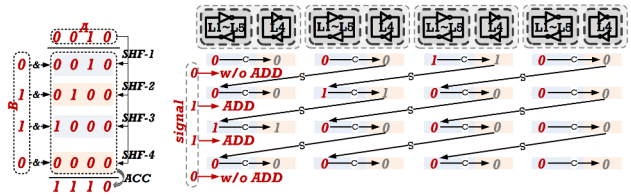


Fig. 12: Spice simulation of basic operation. L1 stores A and L2 stores B initially. The sum result is temporarily stored in L2 at last.



(a) MUL Logic. (b) Multiplication in Ares.

Fig. 13: Multiplication in Ares

In implementing this multiplication method in Ares, the value of operand A is stored in flash memory, while the value of operand B is held in the flash controller. The core principle here is to use the bits of B to decide if an addition operation should follow each shift of A. As shown in figure 13(b), we begin by reading A into L4 of each BL.

The operand B is already held in the controller (B can be input from the host and read from cells). Subsequently, the flash controller issues two types of commands based on each bit of B according to the digit of A (from LSD to MSD). If the current digit of B is *0*, the flash controller sends a *shift* command to flash memory that indicates shift only (w/o ADD in the figure). In this case, Ares enables the transmission circuit to shift the current bit value to the higher digit. It then transfers the bit value in L1 to L4 to prepare for further operations. If the current digit is *1*, the controller will send the *shift_and_add* command. Under this circumstance, Ares performs an addition operation with the current intermediate result. And then activate the transmission circuit to shift the current version of A. During the computation, we use another free latch, L5, to temporarily store the current accumulative result. When performing addition, we transfer value to L2 and save a copy of the current shifted version of A in L1. After the addition, the saved bit value in L5 is transferred to L4 waiting for the next command shift.

Since all chips share the flash controller through a single channel, there is a constraint where one multiplier is shared for parallel multiplications in single plane. Furthermore, multiplication commands for different chips on the same channel are sent in an interleaved manner to enable parallel computation.

G. Ares with Vertical Operand Layout

To further exploit the benefits of BL-level parallelism, we introduce Ares-V, which distributes operands across various WLs vertically sharing the same BL. Ares with conventional data allocation is referred to as Ares-H for clarity. In Ares-V, the vertical layout of operands for addition can be seen in Figure 14(a), where both A and B share the same BL. The addition process involves performing an N-bit full adder by repeatedly executing 1-bit full adders after reading two bits from these operands. When BL-level parallelism is exploited, the Ares-V timeline is illustrated in Figure 14(b). Following steps ❶ to ❺, the sum bit is sent to the controller while the carry-in bit remains for the subsequent 1-bit full adder. This process iterates until all bits are processed. Compared to Ares-

H, Ares-V offers higher BL-level parallelism and eliminates idle times during carry-in propagation.

To perform accumulation in Ares-V, we utilize intra-die transmission to perform divide-and-conquer. In the first addition cycle, all addition operations are simultaneously carried out, similar to the parallel addition operations depicted in Figure 14 (e.g., A + B, C + D, E + F, and so on). Rather than directly outputting the addition results to the controller, an intra-die transfer circuit is employed to transfer these results to the latch circuits within the second plane for the subsequent addition cycle. The design of the intra-die transmission circuit is illustrated in Figure 15, demonstrating the transfer of every two sum results from two latch circuits in Plane 1 to a latch circuit in Plane 2.

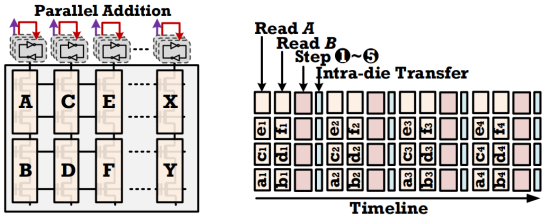
The second addition cycle begins with the calculation of the second digits in Plane 1, which pipeline the first addition with the second one. As the second addition cycle only takes half of the Plane 2 (as in Figure15, two sum results transfer to one BL's latch circuits). During the third addition cycle, the sum results in the first half of Plane 2 are transmitted to the next quarter of the latch circuits within the same plane, preparing them for subsequent addition cycles. This process repeats until all sum results are accumulated together. In this way, each cycle of addition is also pipelined with other cycles. By adopting this approach, Ares-V is performed in die-level parallelism that each group contains two planes. In other words, Ares-V sacrifices plane-level parallelism but gains the benefit of pipelining. It can at most accumulate 256K operands with 16KB page size.

For multiplication, Ares-V stores multiple copies of one multiplier. As in Figure 13(a), a 4-bit data requires 4× space and 4 different shifted versions are stored and could be A C E, and X in Figure 14(a). Then we use Plane 1 to perform bit-multiply (logical `AND`) and accumulate these `AND` results in Plane 2. We notify that Ares-V is proposed for specific applications and requires a dedicated dataset.

V. Ares Implementation

A. System Support

1) Application Requirements: Ares is not universally suitable for all applications, as the efficiency of IFP can only be achieved when a large number of computation operations are required. Thus, applications should determine whether they need Ares computation and determine the data that will be used for Ares computation.



(a) Data Layout. (b) Ares-V Timeline.

Fig. 14: Data Layout and Timeline for Ares-V based Addition.

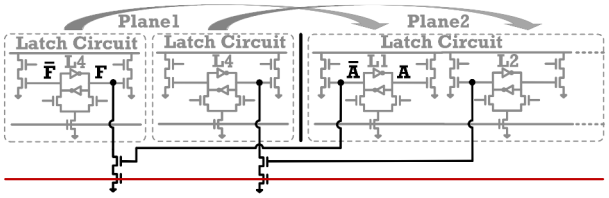


Fig. 15: Intra-die Transmission Circuits.

2) Ares-based API Interface: We provide three main API interfaces integrated into the run-time library for users to interact with Ares-supported SSD: *Ares_write*, *Ares_input* and *Ares_read*. The *Ares_write* interface is used when users write data into SSD. Its parameters include operand size, logical address, layout type, and group_id. The logical address and group_id are essential information provided to tell SSD about the location information of stored data. For example, two operands that need addition should be included in the same group and assigned the same group_id. And the operand size is stored as meta data of a new group. The layout type indicates whether a vertical or horizontal layout is adopted. To simplify the whole system integration and communication, each data persisted by *Ares_write* is composed of multiple operands and aligned with flash physical page size. *Ares_read* retrieve results from SSD with different types of calculations. So far, it provides *add*, *accumulate*, *mul*, and *mul&add*. *Ares_input* is used to send operands to the SSD for multiplication.

The NVMe driver is extended to be aware of Ares operations to parse these API interfaces and send them to the SSD device. Since NVMe command reserves multiple free spaces undefined to any semantic, we can invoke these spaces to store the information of *Ares_write*, *Ares_read* and *Ares_input*. To be more space-efficient, the information, such as logical address and group_id, should still be encapsulated in the original address and size space if the NVMe command has been identified as an Ares-based NVMe command. While this work focuses more on the design and integration, we leave end-to-end efficient system design for future work.

3) SSD Modification: To support Ares in SSD, multiple changes to the SSD are required, including FTL and new flash commands. The SSD controller is responsible for interpreting Ares-based NVMe commands from the host system and converting them into Ares-based device commands with corresponding actions. We divided them into three categories that related to *Ares_write*, *Ares_input* and *Ares_read* separately. Firstly, for *Ares_write*, operands must be programmed according to the specific location requirements of Ares. Ares's FTL allocates PPA depends not only on LPA but also on the group_id that it will try to store the data from the same group_id in the same plane. Subsequently, the standard program command is executed to store the data in the desired location, followed by building the mapping between logical and physical addresses.

Second, when receiving input operand data from *Ares_input*, SSD controller distributes the input data to the corresponding flash controller, waiting for the processing command.

Third, when reading results from Ares-based flash SSD, we design five new flash commands to perform operations according to requests. Commands are: (1) *init*, (2) *add*, (3) *shift*, (4) *shift_and_add*, (5) *end*. Figure 16 gives straightforward examples of new flash command usage under three circumstances. Flash chips accordingly control their page buffer's signal when receiving these commands. Before performing any Ares operations, (1) must be sent to activate Ares mode on the flash chips. It has one bit to indicate whether it is going to Ares-H or Ares-V applications. During the activation of Ares, standard flash read commands followed by address are normally applied between Ares-based flash commands. (5) is sent when all operations are finished and disable Ares mode on the flash chips. As Figure 16 indicates, flash controller sends (2) to perform addition and accumulations. And *add* commands contain 3 bits indicating data types. Depending on each digit of the multiplier (input operand) stored in the flash controller, it correspondingly sends (3) and (4) to perform multiplication. By arranging these commands in different orders, various operations can be combined.

4) Standard Read: When users need to access raw data stored for Ares computation, they can use standard read operations to obtain specific raw data and data chunks in Ares-H. This is because data allocation in Ares-H does not interfere with standard access methods. However, in Ares-V, the dataset is transposed before storage. As a result, users must read the entire transposed data chunk initially. If specific data access is required, the entire data chunk is read, and the target data is buffered. When users access this data subsequently, the Ares-based SSD will reprogram the data using standard data allocation methods.

5) GC in Ares: Since Ares-based SSD stores operands in the same plane, when GC happens, the data is preferred to stay at the same plane. At this moment, it can use copy back [5], [86] and block only one plane [88] to accelerate GC process. By doing this, the data transfer caused by GC is reduced by using copy back command. And the other dies can normally serve I/O since we only block one plane during the GC. Furthermore, it can also work well with wear-leveling at the same time [19].

B. Reliability

To ensure the reliability [17] of Ares, we implement Enhanced-SLC-mode-Programming (ESP) proposed by Flash-Cosmos [72]. It uses a more dedicated programming process to make two voltage states more distinguishable as shown in Figure 17. In other words, it sacrifices programming latency for reading accuracy. We have used an FPGA test board with advanced 3D flash memory to test the bit error of SLC under different P/E cycles. When we perform standard flash programming, the average number of bit errors per page is 0.0653 and 0.147 under 20000 and 40000 P/E cycles. After adopting ESP, we precisely inject charges into cells: we get zero bit errors when the P/E cycle is 40000. Despite the programming latency is enlarged twice, data accuracy is promised and it does not impact standard read/operating latency.

Besides the standard ESP, we also apply ECC as the backup strategy. However, at this time, the ECC is not only to ensure that data is programmed correctly, but also it needs to check the margin between two states in the cell (i.e., robustness). Ares-based SSD will periodically check cell states margin when the SSD is idle. As shown in Figure 17, we use two more reference voltage (V_{ref2} and V_{ref}) to read data. If we find that voltage margin is larger than standard margins, such page will be re-programmed for computing accuracy.

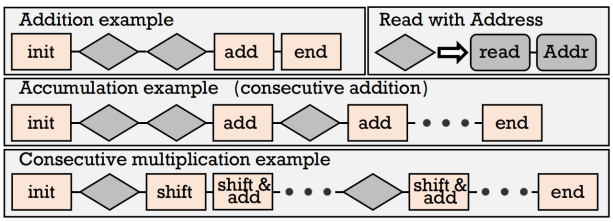


Fig. 16: Flash Commands Usage Examples.

VI. Evaluation

A. Methodology

Evaluated Systems. To assess Ares's efficacy, we examined three distinct computing architectures, which are (*i*) outside-storage processing (OSP), (*ii*) in-storage processing (ISP), and (*iii*) Ares itself. OSP relies on the host CPU for computations, processing data after it's transferred from the SSD. ISP employs an FPGA with 21.2Mb BRAM for both SSD control and computational tasks. The FPGA communicates with the host via an external PCIe connection, sending only computational results after processing. (*iii*) Ares performs operations inside the NAND chip described in Section IV and sends the result only to the SSD controller and then the host application.

Performance Modeling. We utilize the state-of-the-art simulator, MQSim [82], to gauge SSD performance. We employed HSpice for simulating Ares's operational latency at the circuit level, setting each transistor activation at 0.4us, which is referred to [52], [60]. Then, we extend MQSim with Ares functions to faithfully model its performance. We structured the end-to-end throughput analysis of all architectures in two phases. The first is SSD performance, including SSD operations and Ares operations. The second phase is computational throughput collected from the real host CPU and FPGA (measured from the real host system and FPGA testing board). The configurations are listed in Table I.

Energy Modeling. We measure the energy consumption of the host computation by using Intel RAPL [39], and we test

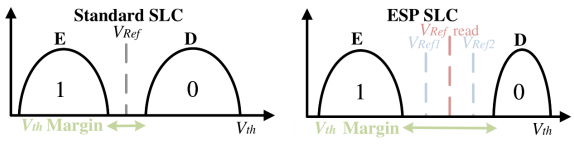


Fig. 17: ESP SLC cell with checking Vref.

TABLE I: Evaluation Configuration

OSP	**CPU**: Intel Xeon Gold 5220; 2.20GHz
	Main Memory: 64 GB; DDR4-3200
	Interface: 8GB/s 4-lane PCIe Gen4
ISP	**FPGA**: Zu3T, 400MHz
	Memory: 21.2Mb BRAM; 4GB DRAM
	Interface: 8GB/s 4-lane PCIe Gen4
SSD	**Bandwidth**: 8GB/s 4-lane PCIe Gen4; 1200MT/s 8-bit internal channel
	SSD: 8 Channels;8 Chips/Channel; 4 Planes/Chip; 16 KB/Page
	Latency: tR: 25 µs; tPROG: 200/700 µs(SLC/TLC)

FPGA energy from the real FPGA board. We collect DRAM power consumption based on the advanced DRAM simulator, DRAMSim3 [62]. For power consumption of SSD, we use commodity SSD power values based on Samsung 990 Pro [3]. Although Ares processing likely requires less power than the entire SSD, we based our calculations for Ares on the maximum power of the SSD for a conservative estimate.

B. Operation Evaluation

Before diving into applications, it's crucial to evaluate the operating latency of Ares's fundamental operation: addition. This evaluation involved using operands in various formats, including *int32*, *int16*, *int8*, and *int4*. We compare Ares with the addition composed by bitwise operation in ParaBit and Flash-Cosmos as depicted in Figure 18. We follow the workflow described in Section IV-D, and each transistor activation phase lasts 400ns [52]. Each bitwise `AND`/`OR` requires three sequential activations in a timeline. Without read latency, the addition per page for Ares-H takes place within 129µs/65µs/33µs/17µs for 32bits/16bits/8bits/4bits respectively. And the addition per page in Ares-V is operated within 10µs for all data types. In both cases, the Ares operation, which utilizes a read operation to acquire operands, increases latency compared to a pure flash read operation. However, it still remains less than the latency of a Single-Level Cell (SLC) program operation. Although ParaBit and Flash-Cosmos can perform high-performance bitwise operations, Ares achieves at least a 160× and 125× speedup compared with them when performing addition. This notable improvement is attributed to the greater flexibility of operations in Ares, which reduces the need for additional reads, programs, and data transfers during the process.

C. Addition Evaluation

Addition is a very common and widely used operation. Here we use vector addition [85] where we range value size varies from *int32* to *int4* to test addition operations. To demonstrate large data scenarios, we set vector size as 200K and change the number of vector additions spanning from 160 to 40960.

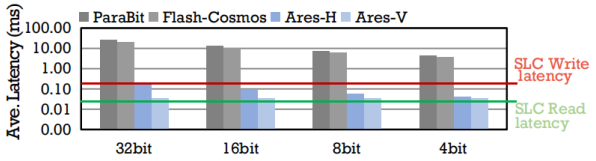


Fig. 18: Addition Latency

The results, as illustrated in Figure 19, demonstrate Ares's superior performance compared to other architectures. Ares-H outperforms OSP by 2.55× and ISP by 1.75× on average, and Ares-V surpasses OSP and ISP by 2.6× and 1.78× separately. These results strongly indicate Ares's efficiency in handling basic, data-intensive computations. The primary advantages of Ares come from the reduced data movement started from NAND flash chips. Ares-V exhibits a slight improvement in performance over Ares-H, even though it has a pipelined workflow with less operating latency per page. This is because the data transfer latency through the internal bus is still longer than the computational latency. The addition evaluation results reveal that Ares achieves high performance while performing addition operations.

D. Accumulation Evaluation

Accumulation is a critical operation in various fields and algorithms for aggregating information. We conducted two case studies assessing the performance of Ares under the accumulation operation.

K-Means Clustering [40] frequently updates the mean values of clusters, necessitating the initial accumulation of all values within the same cluster. When data volume exceeds memory capacity, an out-of-core map-reduce technique is applied to segment the dataset into smaller trunks and store them in storage. When updating final centroids, it reads data from storage first and computes mean values. Ares can first accumulate data inside flash memory and transfer results to the host or SSD controller to calculate the mean. In the evaluation, we varied the dataset size by setting different numbers of trunks (from 32 million to 128 billion), and each trunk contains 1024 values, in formats from *int32* to *int4*.

Page Rank is a widely utilized algorithm in graph analysis, where each node's score is determined by the accumulation of weights from incoming edges. The graph's nature varies under different workloads; hence, we employed real-world graph datasets for evaluation: ljournal-2008 from LiveJournal social site (denoted as `LJ`, $|V|$ = 5.3M, $|E|$ = 79M) [22], enwiki-2022 from a snapshot of the English part of Wikipedia as of mid 2022 (denoted as `WK`, $|V|$ = 6.5M, $|E|$ = 159M) [9], fb_us-2009 from Facebook in 2009 (denoted as `FB9`, $|V|$ = 41M, $|E|$ = 4.6B) [14], and fb_itse-current from Facebook in 2011, (denoted as `FB11`, $|V|$ = 24M, $|E|$ = 5.1B) [13]. For simplicity in testing, we averaged the number of edges (i.e., $|E|$) by the number of vertexes (i.e., $|V|$). For Ares-V, the number of edges was expanded to accommodate its operational needs.

Results analysis: The evaluated results are collected and presented in Figure 20 and Figure 21, respectively. We make three main observations from accumulation experiments.

First, across varying data volumes (MB to GB, extendable to TBs), Ares consistently outperforms OSP and ISP. In the K-means scenario (Figure 20), Ares-H exhibits an average performance improvement of 18.58× by OSP and 9.89× by

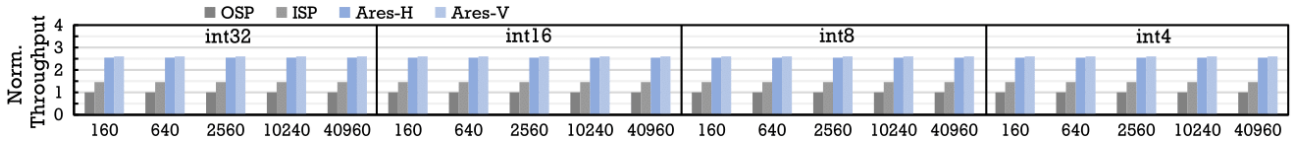


Fig. 19: Vector Addition with Different Number of Vector Groups.

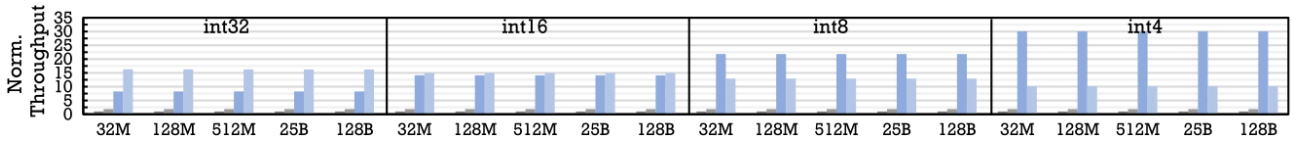


Fig. 20: K-means with Different Number of Trunks.

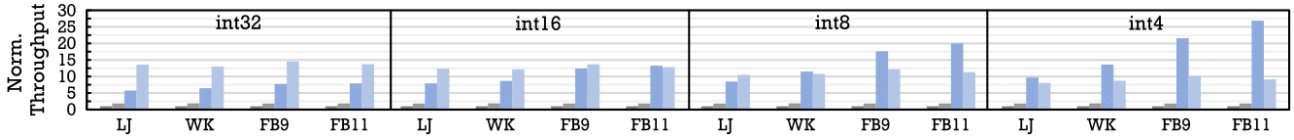


Fig. 21: Page Rank with Various Workloads.

ISP. And Ares-V improves performance by 13.58× and 7.22× on average. For Page Rank results in Figure 21, Ares-H speeds up OSP and ISP by 12.47× and 6.82× on average. And Ares-V outperforms OSP/ISP by 11.66×/6.4× on average. Despite ISP's reduced data movement through external PCIe, the internal data movement inside SSD still hampers its performance. While Ares significantly improves the throughput by reducing data movements from the beginning.

Second, the performance of Ares-H varies from the number of data inside each group when performing accumulation. This is especially obvious when using *int4* as shown in Figure 21. As it accumulates more data values, it gets more benefits by mitigating more data movements from chips to the host.

Third, the performance of Ares varies from different data types. Ares-H shows increased performance as digit length decreases. This is because the operational time in Ares-H shortens when shifting from int32 to int4. But it still offers a minimum of 6×/3.29× performance improvement over OSP/ISP in the worst case (`LJ` when *int32* in Figure 21). Ares-V exhibits smaller performance variations across different data types, performing optimally with *int32*. The reason is that, as operation latency remains constant irrespective of digit size, the proportion of active processing relative to total latency increases with smaller data types.

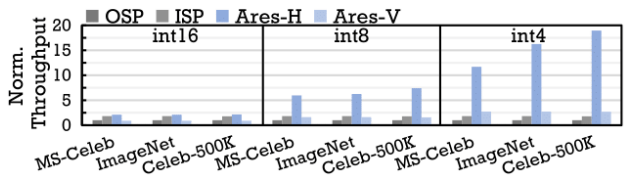


Fig. 22: VVM Performance with Different Workloads.

E. Multiplication Evaluation

Multiplication is also adapted in many important studies like machine learning. We use quantized vector similarity search (VSS) [71] as the case study, which is extremely popular in edge devices for distinguishing faces and other objects (e.g., vector databases). After proper training, each image or object generates a feature vector, and vector-vector multiplication (VVM) is then used to determine the similarity between these feature vectors and input vectors. While VSS requires additional functions post-VVM (e.g., top-k and sorting), we only concentrate on VVM and assume other tasks are handled by computing units. We use vector workloads from MS-celeb [38], ImageNET [29] and Celeb-500k [18] with 10 million, 14 million, and 50 million vectors. Before storage, each vector's number of elements is normalized to 512, and the element format varies from *int16* to *int4*. Ares executes VVM through consecutive multiplications. After each multiplication is finished, the result is retained inside the latch as the current value described in Section IV-F. Ares-V performs VVM by keeping accumulating multiplication results in the second plane.

The VVM results are shown in Figure 22. In this case, although Ares-V sacrifices space to perform multiplication, it matches ISP's performance under *int8* and it still performs 2.74×/1.4× better than OSP/ISP under *int4*. Ares-H outperforms both ISP and OSP for all data types. Under *int16*, it performs 2.12× better than OSP and 1.18× than ISP. The lesser performance gain compared to accumulation is attributed to the longer operation latency for Ares multiplication. As the number of digits decreases, Ares's performance further improves. It speeds up OSP/ISP 7×/3.84× and 16×/8.79× under *int8* and *int4* on average, respectively. Each chip can parallelly perform 128K VVMs with 4-bit quantized data, 64K VVMs with 8-bit data, and 32K VVMs with 16-bit data. For a vector size of 512, the average per-VVM operation per

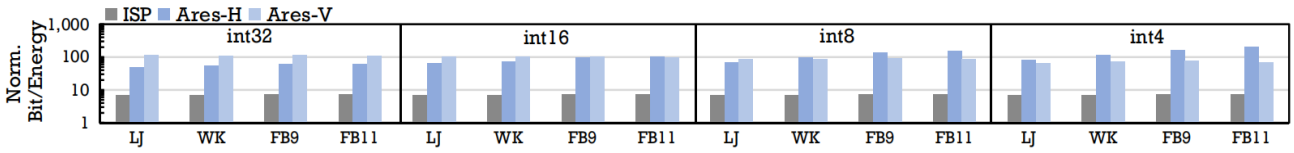


Fig. 23: Energy Efficiency Comparison When Performing Accumulation in Page Rank. Normalized to OSP Already.

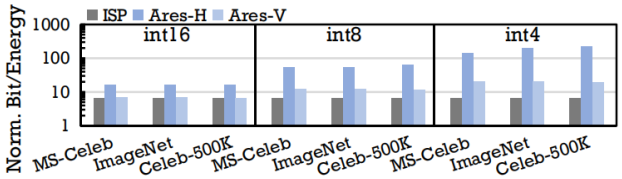


Fig. 24: VVM Energy Evaluation.

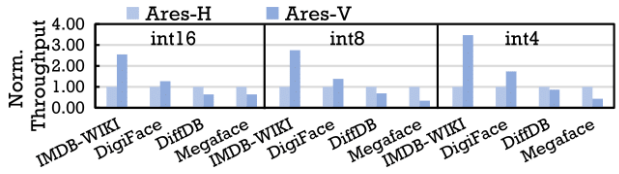


Fig. 25: Ares-H and Ares-V comparison in VVM.

chip is about 0.24µs, 1.26µs, and 8.68µs, respectively, which outperforms 42× the SOTA that performs VVM on edge using flash memory [43](53 µs per VVM/chip w/ 8-bit data).

Multiplication in Ares is not limited to VSS. For example, quantized neural networks, including CNN (Convolutional Neural Networks) [35], [41], [54] and FCNN (Fully Connected Neural Networks). Ares can perform training and inference procedures for quantized models and offload part of processing using multiplication operation [28], [44], [76], [91].

F. Energy Evaluation

Here, we use Page Rank and VSS experiments to evaluate the energy consumption of Ares compared to OSP and ISP.

Page Rank: Figure 23 shows the energy consumption of three architectures when performing accumulation under real-world workloads. The results reveal that Ares is more energy-efficient. Specifically, Ares-H saves, on average, 97.9×/14× more energy than OSP/ISP. Meanwhile, Ares-V achieves an average energy saving of 92.36×/13.2×. These benefits stem from two primary factors: First, the reduced end-to-end latency. For Ares-H, energy savings increase as the number of data operands in each accumulation group grows and as the digits of the operand decrease. For Ares-V, it saves more energy when using the data type of *int32*, which has the best end-to-end throughput in the previous evaluation. Second, the less device power requirements for Ares compared to ISP and OSP. OSP consumes significant energy due to the CPU's dynamic power usage and data movement among SSD, DRAM, and computing units. While ISP, utilizing an energy-efficient FPGA, still expends energy during activation.

VSS: Figure 24 shows up energy consumption when performing vector-vector multiplication. In this case, Ares-H is 89×13.2× times more energy efficient than OSP/ISP. Meanwhile, although Ares-V does not perform as well as accumulation in VSS, it still saves 13×/1.92× more energy than OSP/ISP. These results highlight Ares's high energy efficiency, even in complex operations. Notably, it achieves the best performance when using *int4* that it can save up to 220×/32.6× energy of OSP/ISP.

G. Area Overhead Analysis

Ares leverages NAND flash chips with five latches per BL with extra transmission peripheral to perform operations, while modern flash memory-based SSD already supports up to five and six latches per BL [42], [45], [51], [80]. As the area of the advanced flash die (4 planes with 16KB per page) ranges from 76 mm^2 [51], [75] to 98 mm^2 [42].

Ares's additional area overhead originates from the additional transfer circuit and the modification in the latch circuit. The additional transmission peripheral requires four extra transistors per BL. The modified latch circuit as shown in Figure 6 necessitates two extra transistors per latch (e.g., A and $\overline{A}$ in Fig 6). Given that each transistor occupies an area of (0.4µm × 0.4µm) under 65nm process [52], [60], additional transistors take 1.2%∼1.5% extra area overhead. However, in order to minimize overhead from the modified buffer, we have used only one-side upper latch for all latches in the operations mentioned in Figure 8. Thus, the extra overhead from the page buffer is reduced to be 0, resulting in an additional area overhead of only 0.34%∼0.42%. Considering the pitch must be greater than 130nm in 65nm process [1], we reserve another half of the extra area for wiring, which is more than enough. Then, the Ares will take 0.68%∼0.84% extra area overhead. Compared to the previous work [43], which applied an amount of support logic to form computing units with 1.6% extra overhead, Ares incurs less area overhead and offers higher performance as shown in VI-E.

On the other hand, if we apply extra support logic, it is impossible to achieve high BL-level parallelism with a small area overhead. For example, a 32-bit adder logic requires more than 660 transistors, which means it requires 21 transistors per BL. Additional transistors are also needed for activation, receiving input, and reloading results. The area overhead of such design is far beyond both Ares and [43].

H. Ares and Data Compression

Data compression is commonly used in managing large datasets and can be easily integrated inside the SSD device [4], [49], [92]. Compared with OSP, Ares's execution time is

consistently less than about half of OSP's data transfer time across all examined scenarios. Ares's excution time take only 52% of the standard read of OSP in the worst VVM case and is 10% in the best case. In cumulative studies, this number even drops to 3.8%. This means the Ares is more advanced than OSP when using the widely used lossless compression strategy zstd [27] (compression ratio generally ranges from 2.15 to 2.88). Moreover, OSP's decompression of substantial data volumes introduces much latency, placing considerable strain on the host CPU and memory. For ISP, since the computation units are limited in FPGA, it needs more transfers between SSD DRAM and FPGA for data decompression, which even wrecks the benefits of using ISP.

I. Ares Deployment Scenarios Analysis

In this section, we analyze which types of applications will get benefits from Ares-H and Ares-V.

To begin with, as mentioned earlier, Ares-Flash is designed for big data applications where a vast amount of data is stored in the SSD. Both Ares-H and Ares-V are well-suited for vectorized data with bulk operations. This is especially true for Ares-V, where data is stored as a whole.

For applications that require basic bulk addition and accumulation operations, Ares-V performs better than Ares-H when dealing with larger data types. This means Ares-V is friendly to applications that demand higher precision, while those that prioritize higher speed are better suited for Ares-H.

When applications need to perform multiplications, such as in vector similarity search (VSS), the total data volume and the level of parallelism will determine which option is better. We use the same VSS test as an example, selecting IMDB-WIKI (500K faces) [77], DigiFace-1M (1M faces) [10], Diffusiondb (2M faces) [87], and Megaface (4.7M faces) [50]. As shown in Figure 25, when the dataset contains fewer than 2M identities, using Ares-V is preferable.

Finally, in this work, we suggest that, except for specific applications or scenarios discussed above, Ares-H should be elected as the default option for Ares-Flash due to its consistent performance gains and more user-friendly data allocation.

VII. Related Works

Prior IFP works [24], [37], [47], [53], [66], [69] perform multiplication and accumulation for specific applications like DNNs. Although they achieve high performance using BL-level parallelism, their computation mainly relies on collecting voltage/current, which requires expensive modification to the flash memory (e.g., extra ADC). While Ares achieves accumulation and multiplication with little modification, making it easily adaptable to commodity flash SSDs.

ICE [43] is the first IFP work that achieves 8-bit and 4-bit multiplication and VVM using flash memory without ADC. It applies an extra TC digital accumulator on flash chips to accumulate bit-multiplication results. However, it sacrifices the BL-level parallelism to minimize the area overhead from the extra accumulator. Ares is more efficient than ICE when performing VVM as shown in SectionVI-E. The reason is that, without constraints from extra computing hardware, Ares effectively utilizes BL-level parallelism and can support 16-bit to 4-bit VVM. ParaBit [33] and Flash-Cosmos [72] perform bitwise operations using flash memory with little hardware modification. We have shown that their bitwise operations cannot effectively compose and perform other complex operations in Section III-B. Compared to them, Ares proposed more flexible approaches to perform bitwise operations and efficient control steps to perform arithmetic operations.

VIII. Limitation and Discussion

Floating-point Calculation: Although Ares is scalable for fix-point calculation, it does not fully support floating-point calculation. The main constraint is that all BLs are concurrently operated but floating-point (FP) calculation requires different processing for mantissa and exponent. Fortunately, with some data preprocessing, Ares can still facilitate with some FP calculations. For example, in FP multiplication, we can store mantissa and exponent separately. Then, when performing FP multiplication, Ares can perform addition on its exponent and perform multiplication on its mantissa independently, which forms the final results.

Data Encryption: Like previous IFP works [33], [72], the commonly used encryption algorithms are not suitable for this structure. One solid solution is to use fully-homogeneous encryption algorithms (FHE). FHE allows a broad spectrum of computations to be carried out on ciphertexts, producing an outcome that, once decrypted, matches the result of operations performed on the plaintext. Currently, there are many FHE works being widely studied and applied [15], [21], [23]. Some neural-network studies [26], [59] have already used CKKS [21] and TFHE [23] and have proved its practicable.

IX. Conclusion

In this paper, we proposed Ares, a new IFP architecture to perform arithmetic operations, including addition, accumulation, and multiplication using flash memory. And other operations can be composed by proposed operations. Evaluations show that Ares is extraordinarily efficient in massive data volume workloads while both the data transfer overhead and the energy spent on the whole system are significantly decreased compared to existing architectures.

Acknowledgment

We would like to thank anonymous reviewers for their valuable comments to this work. We also thank Qing Wang, Yunpeng Li, and Jian Gao for their suggestion. This work is supported by the National Natural Science Foundation of China (Grant No. U22B2023), NSFC (No.62102219), and the Fundamental Research Funds for the Central Universities (No.20720230071). Jiwu Shu (shujw@tsinghua.edu.cn) and Congming Gao (gaocm@xmu.edu.cn) are the corresponding authors.

REFERENCES

[1] "International technology roadmap for semiconductors," https://web.archive.org/web/20070927025913/http://www.itrs.net/Links/2006Update/FinalToPost/04_PIDS2006Update.pdf.

[2] "Pcie specification," https://pcisig.com/specifications/pciexpress/.

[3] "Samsung, samsung 990 pro ssd," https://semiconductor.samsung.com/consumer-storage/internal-ssd/990-pro/.

[4] "Data compression in the intel® solid-state drive 520 series," https://www.intel.com/content/dam/www/public/us/en/documents/technology-briefs/ssd-520-tech-brief.pdf, 2012.

[5] "Micron, 256gib to 1tib async/sync nand features," https://www.micron.com/content/dam/micron/global/public/spectek/data-sheet/nand-flash/tlc/spectek-256gb-1tb-nand-tlc-a-sync-b16a.pdf, 2017.

[6] "Open nand flash interface specification revision 5.1," https://www.onfi.org/specifications, 2022.

[7] S. Aga, S. Jeloka, A. Subramaniyan, S. Narayanasamy, D. Blaauw, and R. Das, "Compute caches," in *HPCA*, 2017.

[8] S. Agatonovic-Kustrin and R. Beresford, "Basic concepts of artificial neural network (ann) modeling and its application in pharmaceutical research," *JPBA*, 2000.

[9] L. Backstrom, P. Boldi, M. Rosa, J. Ugander, and S. Vigna, "Four degrees of separation," in *WEBSCI*, 2012.

[10] G. Bae, M. de La Gorce, T. Baltrušaitis, C. Hewitt, D. Chen, J. Valentin, R. Cipolla, and J. Shen, "Digiface-1m: 1 million digital face images for face recognition," in *WACV*, 2023.

[11] J. Bae, J. Lee, Y. Jin, S. Son, S. Kim, H. Jang, T. J. Ham, and J. W. Lee, "{FlashNeuron}:{SSD-Enabled}{Large-Batch} training of very deep neural networks," in *FAST*, 2021.

[12] M. Bavandpour, S. Sahay, M. R. Mahmoodi, and D. Strukov, "Efficient mixed-signal neurocomputing via successive integration and rescaling," *VLSI*, 2019.

[13] P. Boldi, M. Rosa, M. Santini, and S. Vigna, "Layered label propagation: A multiresolution coordinate-free ordering for compressing social networks," in *Proceedings of the 20th international conference on World Wide Web*, S. Srinivasan, K. Ramamritham, A. Kumar, M. P. Ravindra, E. Bertino, and R. Kumar, Eds., 2011.

[14] P. Boldi and S. Vigna, "The WebGraph framework I: Compression techniques," in *WWW*, 2004.

[15] Z. Brakerski, C. Gentry, and V. Vaikuntanathan, "(leveled) fully homomorphic encryption without bootstrapping," *TOCT*, 2014.

[16] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," in *NeurIPS*, 2020.

[17] Y. Cai, S. Ghose, E. F. Haratsch, Y. Luo, and O. Mutlu, "Error characterization, mitigation, and recovery in flash-memory-based solid-state drives," *Proceedings of the IEEE*, 2017.

[18] J. Cao, Y. Li, and Z. Zhang, "Celeb-500k: A large training dataset for face recognition," in *ICIP*, 2018.

[19] L.-P. Chang and C.-D. Du, "Design and implementation of an efficient wear-leveling algorithm for solid-state-disk microcontrollers," *TODAES*, 2009.

[20] F. Chen, R. Lee, and X. Zhang, "Essential roles of exploiting internal parallelism of flash memory based solid state drives in high-speed data processing," in *HPCA*, 2011.

[21] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *ASIACRYPT*, 2017.

[22] F. Chierichetti, R. Kumar, S. Lattanzi, M. Mitzenmacher, A. Panconesi, and P. Raghavan, "On compressing social networks," in *SIGKDD*, 2009.

[23] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachene, "Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds," in *ASIACRYPT*, 2016.

[24] W. H. Choi, P.-F. Chiu, W. Ma, G. Hemink, T. T. Hoang, M. Lueker-Boden, and Z. Bandic, "An in-flash binary neural network accelerator with slc nand flash array," in *ISCAS*, 2020.

[25] L. Chua, "Memristor-the missing circuit element," *IEEE Trans. Circuit Theory.*, 1971.

[26] P.-E. Clet, O. Stan, and M. Zuber, "Bfv, ckks, tfhe: Which one is the best for a secure neural network evaluation in the cloud?" in *ACNS*, 2021.

[27] Y. Collet and M. Kucherawy, "Zstandard compression and the application/zstd media type," Tech. Rep., 2018.

[28] M. Courbariaux, I. Hubara, D. Soudry, R. El-Yaniv, and Y. Bengio, "Binarized neural networks: Training deep neural networks with weights and activations constrained to+ 1 or-1," *arXiv*, 2016.

[29] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *CVPR*, 2009.

[30] R. Fan, M. Xie, H. Jiang, and Y. Lu, "Maxembed: Maximizing ssd bandwidth utilization for huge embedding models serving," in *ASPLOS*, 2024.

[31] J. D. Ferreira, G. Falcao, J. Gómez-Luna, M. Alser, L. Orosa, M. Sadrosadati, J. S. Kim, G. F. Oliveira, T. Shahroodi, A. Nori *et al.*, "pluto: Enabling massively parallel computation in dram via lookup tables," in *MICRO*, 2022.

[32] C. Gao, L. Shi, C. Ji, Y. Di, K. Wu, C. J. Xue, and E. H.-M. Sha, "Exploiting parallelism for access conflict minimization in flash-based solid state drives," *TCAD*, 2017.

[33] C. Gao, X. Xin, Y. Lu, Y. Zhang, J. Yang, and J. Shu, "Parabit: processing parallel bitwise operations in nand flash memory based ssds," in *MICRO*, 2021.

[34] C. Gao, M. Ye, Q. Li, C. J. Xue, Y. Zhang, L. Shi, and J. Yang, "Constructing large, durable and fast ssd system via reprogramming 3d tlc flash memory," in *MICRO*, 2019.

[35] R. Girshick, "Fast r-cnn," in *ICCV*, 2015.

[36] T. Grunzke, "Onfi 3.0: The path to 400mt/s nand interface speeds," *Flash Memory Summit*, 2010.

[37] B. Gu, A. S. Yoon, D.-H. Bae, I. Jo, J. Lee, J. Yoon, J.-U. Kang, M. Kwon, C. Yoon, S. Cho *et al.*, "Biscuit: A framework for near-data processing of big data workloads," *SIGARCH*, 2016.

[38] Y. Guo, L. Zhang, Y. Hu, X. He, and J. Gao, "Ms-celeb-1m: A dataset and benchmark for large-scale face recognition," in *ECCV*, 2016.

[39] M. Hähnel, B. Döbel, M. Völp, and H. Härtig, "Measuring energy consumption for short code paths using rapl," *SIGMETRICS*, 2012.

[40] J. A. Hartigan and M. A. Wong, "Algorithm as 136: A k-means clustering algorithm," *JRSS series c*, 1979.

[41] K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask r-cnn," in *ICCV*, 2017.

[42] T. Higuchi, T. Kodama, K. Kato, R. Fukuda, N. Tokiwa, M. Abe, T. Takagiwa, Y. Shimizu, J. Musha, K. Sakurai *et al.*, "30.4 a 1tb 3b/cell 3d-flash memory in a 170+ word-line-layer technology," in *ISSCC*, 2021.

[43] H.-W. Hu, W.-C. Wang, Y.-H. Chang, Y.-C. Lee, B.-R. Lin, H.-M. Wang, Y.-P. Lin, Y.-M. Huang, C.-Y. Lee, T.-H. Su *et al.*, "Ice: An intelligent cognition engine with 3d nand-based in-memory computing for vector similarity search acceleration," in *MICRO*, 2022.

[44] I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, and Y. Bengio, "Binarized neural networks," in *NeurIPS*, 2016.

[45] H. Huh, W. Cho, J. Lee, Y. Noh, Y. Park, S. Ok, J. Kim, K. Cho, H. Lee, G. Kim *et al.*, "13.2 a 1tb 4b/cell 96-stacked-wl 3d nand flash memory with 30mb/s program throughput using peripheral circuit under memory cell array technique," in *ISSCC*, 2020.

[46] S.-W. Jun, M. Liu, S. Lee, J. Hicks, J. Ankcorn, M. King, and S. Xu, "Bluedbm: An appliance for big data analytics," *SIGARCH*, 2015.

[47] S.-W. Jun, A. Wright, S. Zhang, S. Xu *et al.*, "Grafboost: Using accelerated flash storage for external graph analytics," in *ISCA*, 2018.

[48] M. Jung and M. T. Kandemir, "Sprinkler: Maximizing resource utilization in many-chip solid state disks," in *HPCA*, 2014.

[49] M. Kang, W. Lee, J. Kim, and S. Kim, "Pr-ssd: Maximizing partial read potential by exploiting compression and channel-level parallelism," *IEEE TC*, 2022.

[50] I. Kemelmacher-Shlizerman, S. M. Seitz, D. Miller, and E. Brossard, "The megaface benchmark: 1 million faces for recognition at scale," in *CVPR, year=2016*.

[51] A. Khakifirooz, S. Balasubrahmanyam, R. Fastow, K. H. Gaewsky, C. W. Ha, R. Haque, O. W. Jungroth, S. Law, A. S. Madraswala, B. Ngo *et al.*, "30.2 a 1tb 4b/cell 144-tier floating-gate 3d-nand flash memory with 40mb/s program throughput and 13.8 gb/mm 2 bit density," in *ISSCC*, 2021.

[52] M. Kim, S. W. Yun, J. Park, H. K. Park, J. Lee, Y. S. Kim, D. Na, S. Choi, Y. Song, J. Lee *et al.*, "A 1tb 3b/cell 8th-generation 3d-nand flash memory with 164mb/s write throughput and a 2.4 gb/s interface," in *ISSCC*, 2022.

[53] S. Kim, Y. Jin, G. Sohn, J. Bae, T. J. Ham, and J. W. Lee, "Behemoth: A flash-centric training accelerator for extreme-scale dnns." in *FAST*, 2021.

[54] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *CACM*, 2017.

[55] S. Kvatinsky, D. Belousov, S. Liman, G. Satat, N. Wald, E. G. Friedman, A. Kolodny, and U. C. Weiser, "Magic—memristor-aided logic," *TCAS-II*, 2014.

[56] S. Kvatinsky, A. Kolodny, U. C. Weiser, and E. G. Friedman, "Memristor-based imply logic design procedure," in *ICCD*, 2011.

[57] S. Kvatinsky, G. Satat, N. Wald, E. G. Friedman, A. Kolodny, and U. C. Weiser, "Memristor-based material implication (imply) logic: Design principles and methodologies," *VLSI*, 2013.

[58] J. H. Lee, H. Zhang, V. Lagrange, P. Krishnamoorthy, X. Zhao, and Y. S. Ki, "Smartssd: Fpga accelerated near-storage data analytics on ssd," *IEEE Computer architecture letters*, 2020.

[59] J.-W. Lee, E. Lee, Y. Lee, Y.-S. Kim, and J.-S. No, "High-precision bootstrapping of rns-ckks homomorphic encryption using optimal minimax polynomial approximation and inverse sine function," in *EUROCRYPT*, 2021.

[60] T. Lee, I.-J. Jung, and S.-O. Jung, "High-precision and low-power offset canceling tristate sensing latch in nand flash memory," *TCAS-II*, 2023.

[61] F. Li, Y. Lu, Z. Wu, and J. Shu, "Ascache: An approximate ssd cache for error-tolerant applications," in *DAC*, 2019.

[62] S. Li, Z. Yang, D. Reddy, A. Srivastava, and B. Jacob, "Dramsim3: A cycle-accurate, thermal-capable dram simulator," *IEEE Computer Architecture Letters*, 2020.

[63] S. Li, D. Niu, K. T. Malladi, H. Zheng, B. Brennan, and Y. Xie, "Drisa: A dram-based reconfigurable in-situ accelerator," in *MICRO*, 2017.

[64] S. Li, C. Xu, Q. Zou, J. Zhao, Y. Lu, and Y. Xie, "Pinatubo: A processing-in-memory architecture for bulk bitwise operations in emerging non-volatile memories," in *DAC*, 2016.

[65] S. Li, F. Tu, L. Liu, J. Lin, Z. Wang, Y. Kang, Y. Ding, and Y. Xie, "Ecssd: Hardware/data layout co-designed in-storage-computing architecture for extreme classification," in *ISCA*, 2023.

[66] H.-T. Lue, P.-K. Hsu, M.-L. Wei, T.-H. Yeh, P.-Y. Du, W.-C. Chen, K.-C. Wang, and C.-Y. Lu, "Optimal design methods to transform 3d nand flash into a high-density, high-bandwidth and low-power nonvolatile computing in memory (nvcim) accelerator for deep-learning neural networks (dnn)," in *IEDM*, 2019.

[67] V. S. Mailthody, Z. Qureshi, W. Liang, Z. Feng, S. G. De Gonzalo, Y. Li, H. Franke, J. Xiong, J. Huang, and W.-m. Hwu, "Deepstore: In-storage acceleration for intelligent queries," in *MICRO*, 2019.

[68] H. Mao, J. Shu, M. Song, and T. Li, "Lrgan: A compact and energy efficient pim-based architecture for gan training," *IEEE TC*, 2020.

[69] F. Merrikh-Bayat, X. Guo, M. Klachko, M. Prezioso, K. K. Likharev, and D. B. Strukov, "High-performance mixed-signal neurocomputing with nanoscale floating-gate memory cell arrays," *TNNLS*, 2017.

[70] R. Micheloni, L. Crippa, and A. Marelli, *Inside NAND flash memories*, 2010.

[71] M. Norouzi, D. J. Fleet, and R. R. Salakhutdinov, "Hamming distance metric learning," in *NeurIPS*, 2012.

[72] J. Park, R. Azizi, G. F. Oliveira, M. Sadrosadati, R. Nadig, D. Novo, J. Gómez-Luna, M. Kim, and O. Mutlu, "Flash-cosmos: In-flash bulk bitwise operations using inherent computation capability of nand flash memory," in *MICRO*, 2022.

[73] K.-T. Park, M. Kang, D. Kim, S.-W. Hwang, B. Y. Choi, Y.-T. Lee, C. Kim, and K. Kim, "A zeroing cell-to-cell interference page architecture with temporary lsb storing and parallel msb program scheme for mlc nand flash memories," *JSSC*, 2008.

[74] S. K. Park, Y. Park, G. Shim, and K. H. Park, "Cave: Channel-aware buffer management scheme for solid state disk," in *SAC*, 2011.

[75] T. Pekny, L. Vu, J. Tsai, D. Srinivasan, E. Yu, J. Pabustan, J. Xu, S. Deshmukh, K.-F. Chan, M. Piccardi *et al.*, "A 1-tb density 4b/cell 3d-nand flash on 176-tier technology with 4-independent planes for read using cmos-under-the-array," in *ISSCC*, 2022.

[76] M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi, "Xnor-net: Imagenet classification using binary convolutional neural networks," in *ECCV*, 2016.

[77] R. Rothe, R. Timofte, and L. Van Gool, "Deep expectation of real and apparent age from a single image without facial landmarks," *IJCV*, 2018.

[78] S. Sagiroglu and D. Sinanc, "Big data: A review," in *CTS*, 2013.

[79] V. Seshadri, D. Lee, T. Mullins, H. Hassan, A. Boroumand, J. Kim, M. A. Kozuch, O. Mutlu, P. B. Gibbons, and T. C. Mowry, "Ambit: In-memory accelerator for bulk bitwise operations using commodity dram technology," in *MICRO*, 2017.

[80] N. Shibata, K. Kanda, T. Shimizu, J. Nakai, O. Nagao, N. Kobayashi, M. Miakashi, Y. Nagadomi, T. Nakano, T. Kawabe *et al.*, "A 1.33-tb 4-bit/cell 3-d flash memory on a 96-word-line-layer technology," *JSSC*, 2019.

[81] J. Shu, K. Fang, Y. Chen, and S. Wang, "Th-issd: Design and implementation of a generic and reconfigurable near-data processing framework," *ACM TECS*, 2023.

[82] A. Tavakkol, J. Gómez-Luna, M. Sadrosadati, S. Ghose, and O. Mutlu, "Mqsim: A framework for enabling realistic studies of modern multi-queue {SSD} devices," in *FAST*, 2018.

[83] M. Torabzadehkashi, S. Rezaei, A. Heydarigorji, H. Bobarshad, V. Alves, and N. Bagherzadeh, "Catalina: in-storage processing acceleration for scalable big data analytics," in *PDP*, 2019.

[84] M. Jung and M. T. Kandemir, "An evaluation of different page allocation strategies on high-speed ssds." in *HotStorage*, 2012.

[85] Q. Wang, X. Zhang, Y. Zhang, and Q. Yi, "Augem: automatically generate high performance dense linear algebra kernels on x86 cpus," in *SC*, 2013.

[86] W. Wang and T. Xie, "Pcftl: A plane-centric flash translation layer utilizing copy-back operations," *TPDS*, 2014.

[87] Z. J. Wang, E. Montoya, D. Munechika, H. Yang, B. Hoover, and D. H. Chau, "Diffusiondb: A large-scale prompt gallery dataset for text-to-image generative models," *arXiv*, 2022.

[88] S. Yan, H. Li, M. Hao, M. H. Tong, S. Sundararaman, A. A. Chien, and H. S. Gunawi, "Tiny-tail flash: Near-perfect elimination of garbage collection tail latencies in nand ssds," *TOS*, 2017.

[89] Y. Zhang, Z. Jia, H. Du, R. Xue, Z. Shen, and Z. Shao, "A practical highly paralleled reram-based dnn accelerator by reusing weight pattern repetitions," *TCAD*, 2021.

[90] Y. Zhang, Z. Jia, Y. Pan, H. Du, Z. Shen, M. Zhao, and Z. Shao, "Pattpim: A practical reram-based dnn accelerator by reusing weight pattern repetitions," in *DAC*, 2020.

[91] S. Zhu, L. H. Duong, and W. Liu, "Xor-net: An efficient computation pipeline for binary neural network inference on edge devices," in *ICPADS*, 2020.

[92] A. Zuck, S. Toledo, D. Sotnikov, and D. Harnik, "Compression and {SSDs}: Where and how?" in *INFLOW 14*, 2014.